

MODUL BINAAN & CHEATSHEET RESMI

PANDUAN FUNDAMENTAL FULLSTACK DEVELOPER

Membangun Pemahaman Arsitektur Software Modern dari Nol
Berdasarkan Studi Kasus Nyata Aplikasi SIMMACI

Disusun Khusus Untuk: Pengembangan Diri & Penguasaan Arsitektur SIMMACI

Studi Kasus: React 19 + TypeScript + Laravel 12 + PostgreSQL

Tingkat: Pemula Tanpa Dasar Coding (Konseptual hingga Praktikal)

Waktu Penyusunan: September 2026

BAB 1: Peta Besar Fullstack (Analogi Restoran)

Banyak pemula merasa terintimidasi oleh coding karena menganggap software adalah mantra misterius. Sebenarnya, seluruh aplikasi website modern di dunia (dari Facebook, Tokopedia, hingga SIMMACI) bekerja persis seperti sebuah **Restoran** yang melayani pelanggan.

4 Pilar Utama Arsitektur Fullstack:

1. **Frontend (Meja Pelanggan):** Tampilan yang dilihat dan disentuh pengguna di browser (tombol, formulir input, warna, tata letak). Di SIMMACI dibangun menggunakan *React & Tailwind CSS*.
2. **API / Internet (Pelayan Restoran):** Kurir pembawa pesan lewat kabel internet menggunakan protokol HTTP. Membawa formulir pesanan dan membawa kembali makanan dalam format standar bernama *JSON*.
3. **Backend (Dapur & Koki):** Komputer server yang aman dan tidak bisa dilihat orang luar. Memeriksa izin hak akses (otorisasi), memvalidasi kelayakan data, dan memasak data. Di SIMMACI menggunakan *Laravel (PHP)*.
4. **Database (Gudang & Kulkas Bahan):** Harddisk penyimpanan data terstruktur permanen. Di SIMMACI menggunakan *PostgreSQL / MySQL*.

Siklus 7 Langkah Permintaan Data (End-to-End Request Lifecycle)

Ketika seorang operator sekolah menekan tombol **"Simpan Data"** di layar laptopnya, berikut 7 tahap yang terjadi dalam sepersekian detik:

1. **User Trigger (Frontend):** Jari mengklik tombol ? Komponen React menangkap aksi klik (*onClick*).
2. **Mengirim Paket (API Request):** Data dibungkus ke format JSON dan dikirim oleh Axios/Fetch via metode *HTTP POST*.
3. **Resepsionis Gerbang (Routing):** Server Laravel menerima paket dan mengarahkannya ke koki yang bertugas (*Controller*).
4. **Pemeriksaan & Keamanan (Validation & Authorization):** Koki memeriksa apakah data lengkap (*Validation*) dan apakah pengguna berhak mengubah data tersebut (*Security check*).
5. **Menyimpan ke Gudang (Database Query):** Jika lolos sensor, data dimasukkan permanen ke tabel database via *Eloquent Model*.
6. **Mengirim Balasan (API Response):** Koki memberikan nota kepada pelayan bertuliskan `{"success": true}`.

7. **Umpan Balik Layar (Reactivity):** Browser menerima nota ? React otomatis memunculkan notifikasi hijau (Toast) dan memperbarui tabel di layar secara instan tanpa perlu memuat ulang halaman.

BAB 2: Anatomi Bahasa Coding (Baku vs Bebas)

Salah satu ketakutan terbesar pemula adalah: "Apakah bahasa koding itu baku dan kaku?" Jawabannya: **YA** pada tata bahasanya (sintaks), tetapi **BEBAS** pada penamaannya.

Mengapa Komputer Menuntut Bahasa Baku?

Manusia memiliki akal sehat. Jika Anda membaca pesan: "Sy^a mw mkan", Anda tetap paham maksudnya. Namun komputer **tidak memiliki akal sehat**. Komputer adalah mesin hitung yang luar biasa cepat namun buta konteks. Jika Anda salah mengetik satu huruf saja (misalnya menulis retrun alih-alih return), atau lupa tanda titik koma (;), komputer akan langsung mogok kerja dan mengeluarkan status **Syntax Error**.

Perbandingan Bagian Baku vs Bagian Bebas

Kategori	Contoh di SIMMACI	Aturan Pemakaian
Kata Kunci Baku (Keywords)	public, function, if, return, try, catch	Bawaan resmi bahasa PHP / JavaScript. Wajib diketik persis hurufnya, tidak boleh diganti.
Tanda Baca Baku (Syntax)	Tanda kurung (), kurung kurawal {}, titik koma ;, panah ->	Tanda baca pembuka dan penutup instruksi. Tidak boleh tertinggal atau salah pasangan.
Nama Variabel (Bebas)	\$user, \$schoolId, \$namaGuru	Programmer bebas memberi nama wadah penyimpanan data asalkan didahului simbol \$.
Nama Fungsi (Bebas)	isSensitiveKey(), handleLogin()	Bebas dikarang oleh programmer untuk menggambarkan aksi yang dilakukan fungsi tersebut.
Teks String (Bebas)	"Akses ditolak!", "Login Berhasil!"	Bebas ditulis dalam bahasa Indonesia, Inggris, atau bahasa apapun karena untuk dibaca manusia.

BAB 3: Database (Pondasi Penyimpanan Permanen)

Database adalah hal paling mendasar dalam rekayasa perangkat lunak. Tanpa data, frontend secantik apapun tidak memiliki makna.

Mental Model: Database Sebagai Kumpulan Sheet Excel

Cara termudah membayangkan database relasional (seperti PostgreSQL di SIMMACI) adalah seperti sebuah file **Microsoft Excel**:

- Satu file Excel besar = **Database**.
- Satu lembar kerja (Sheet) = **Tabel (Table)**, misalnya sheet *teachers*, *schools*, *settings*.
- Nama kolom di baris paling atas (Header) = **Field / Column**, misalnya kolom *id*, *nama*, *created_at*.
- Baris data nomor 1, 2, 3 ke bawah = **Data / Baris (Record / Row)**.

Mengenal "Migration": Cetak Biru Otomatis

Developer profesional tidak membuat tabel dengan cara klik mouse di phpMyAdmin. Di Laravel, kita menulis kertas cetak biru bernama **Migration**. Tujuannya: agar database bisa dibangun otomatis di laptop mana saja atau di server cloud dalam hitungan detik.

Contoh nyata dari file cetak biru SIMMACI:

```
Schema::create('settings', function (Blueprint $table) {  
    $table->id(); // Nomor urut unik otomatis (1, 2, 3...)   
    $table->string('key'); // Kolom teks pendek untuk nama pengaturan   
    $table->text('value')->nullable(); // Kolom teks panjang, BOLEH KOSONG (nullable)   
    $table->foreignId('school_id')->nullable(); // Kunci relasi yang menyambung ke tabel schools   
    $table->timestamps(); // Otomatis membuat 2 kolom: created_at & updated_at   
});
```

Relasi Antar-Tabel (Foreign Key) & Integritas Data

Mengapa database disebut **RDBMS (Relational Database)**? Karena tabel-tabel saling dihubungkan menggunakan kunci `foreignId`. Misalnya, data Guru tidak perlu mengetik nama sekolahnya berulang kali, cukup mencantumkan nomor `school_id: 1`. Ini menghemat ruang, mencegah salah ketik, dan mempermudah update serentak.

Studi Kasus Kritis: `cascadeOnDelete` vs `nullOnDelete`

Apa yang terjadi jika sebuah data **Sekolah** dihapus dari sistem?

- `cascadeOnDelete()` (**Efek Air Terjun**): Semua data anak yang terikat *IKUT TERHAPUS PERMANEN* dari harddisk. Cocok untuk data yang tidak bernilai tanpa induknya (misal: arsip SK sementara).
- `nullOnDelete()` (**Dikosongkan Hubungannya**): Data anak *TETAP UTUH* tersimpan, hanya kolom `school_id`-nya yang diubah menjadi kosong (`null`).

Keputusan Arsitektur: Pada tabel *Absensi Guru*, kita **WAJIB** memilih `nullOnDelete()` demi **keperluan audit sejarah & hukum**, agar riwayat kerja guru di masa lalu tidak lenyap saat restrukturisasi sekolah.

BAB 4: Eloquent Model (Juru Bicara Database dalam MVC)

Setelah tabel di database tercipta, bagaimana cara Koki (Controller) berbicara dengan tabel tersebut? Apakah harus mengetik query database SQL rumit? **TIDAK PERLU.**

Laravel menyediakan jembatan yang disebut **Model (Eloquent ORM)**. Setiap tabel memiliki satu file Model sebagai perwakilan resminya.

Anatomi Model Setting (Hanya 33 Baris!)

Perhatikan contoh elegan dari file backend/app/Models/Setting.php:

```
class Setting extends Model
{
    // 1. SATPAM PINTU KOLOM (Mass Assignment Protection)
    protected $fillable = ['key', 'value', 'storage_id', 'mime_type', 'school_id'];

    // 2. PENJELASAN HUBUNGAN KELUARGA DENGAN BAHASA MANUSIA
    public function school()
    {
        return $this->belongsTo(School::class);
    }
}
```

Dua Konsep Terpenting di Model:

1. **Satpam \$fillable:** Daftar kolom mana saja yang diizinkan untuk diisi oleh formulir pengguna dari web. Kolom lain yang tidak terdaftar akan otomatis ditolak oleh Laravel demi mencegah serangan peretasan (*Mass Assignment Vulnerability*).
2. **Bahasa Relasi Manusiawi:**
 - `belongsTo(School::class)`: Menyatakan bahwa satu data Pengaturan adalah *milik dari* Sekolah.
 - `hasMany(Teacher::class)`: Di file `School.php`, menyatakan bahwa satu Sekolah *memiliki banyak* Guru.

Mengenal Pola Desain MVC (Model - View - Controller):

MVC adalah pola arsitektur software paling populer di dunia industri:

- **M (Model):** Mengelola data dan aturan database (contoh: `Setting.php`, `Teacher.php`).
- **V (View):** Tampilan antarmuka yang dilihat pengguna di monitor (komponen React di folder `src/`).
- **C (Controller):** Pengatur logika lalu lintas data (contoh: `SettingController.php`).

BAB 5: Backend Controller (Koki Logika & Keamanan)

Controller adalah ruang dapur tempat semua logika bisnis dijalankan sebelum data diizinkan masuk ke database.

Bedah Kode Nyata: SettingController.php

Berikut potongan baris 57–76 dari file `SettingController.php` yang kita pelajari bersama:

```
public function store(Request $request): JsonResponse
{
    // TAHAP 1: VALIDASI DATA MASUK
    $request->validate([
        'key' => 'required|string|max:255',
        'value' => 'nullable',
    ]);

    // TAHAP 2: KEAMANAN ROLE & OTORISASI
    $user = $request->user();
    if ($user?->isOperator() && $this->isSensitiveKey($request->key)) {
        abort(403, 'Akses ditolak: Operator tidak dapat mengubah konfigurasi rahasia.');
```

Penjelasan Logika Langkah demi Langkah:

1. Validasi (`validate`):

- `required` = Wajib ada isinya (tidak boleh kosong).
- `string` = Harus berupa teks/huruf.
- `max:255` = Panjang huruf maksimal 255 karakter.
- `nullable` = Nilai value boleh dikosongkan.

2. Otorisasi (`if && abort`):

Kondisi logika gabungan menggunakan operator `&&` (DAN). Jika orang yang login adalah Operator *DAN* kunci yang disentuh adalah kunci rahasia (PIN/Token WhatsApp), maka sistem langsung mengusirnya dengan kode **403 Forbidden**. Sebaliknya, Super Admin lolos

tanpa hambatan.

3. **Multi-Tenancy Scoping:** Operator sekolah otomatis dipatok hanya bisa mengelola ID sekolahnya sendiri (\$user->school_id). Ini mencegah Operator SMP 1 mengutak-atik data milik SMP 2.

BAB 6: Frontend dengan React & TypeScript

(Layar Interaktif)

Frontend bertanggung jawab atas apa yang tampak di layar kaca monitor pengguna dan bagaimana antarmuka merespons sentuhan fisik pengguna.

Tiga Pilar Tampilan Web:

- **HTML (Kerangka / Tulang):** Menentukan komponen apa saja yang ada (misal: `<Input />` untuk kotak ketik dan `<Button>` untuk tombol).
- **CSS / Tailwind (Baju & Riasan):** Menentukan warna hijau gradasi Ma'arif, efek kaca blur (*glassmorphism*), bayangan, dan font.
- **React / JavaScript (Otot & Otak):** Bereaksi saat tombol diklik, mengingat data sementara, dan berbicara dengan backend.

Konsep Sakti Frontend: "STATE" (Ingatan Komponen)

Di dunia React, kemampuan aplikasi untuk "**mengingat kondisi terkini di layar**" disebut **State** (dibuat menggunakan fungsi `useState`).

Perhatikan contoh nyata pada tombol mata intip password di `src/features/auth/LoginPage.tsx`:

```
// 1. Komponen mengingat: apakah password sedang diintip atau disembunyikan?
const [showPassword, setShowPassword] = useState(false);

// 2. Kotak input berganti mode otomatis sesuai State:
<Input
  type={showPassword ? "text" : "password"}
  placeholder="***"
/>

// 3. Tombol mata membalik kondisi State saat diklik:
<button onClick={() => setShowPassword(!showPassword)}>
  {showPassword ? <EyeOff /> : <Eye />}
</button>
```

Tanda seru (`!showPassword`) artinya *kebalikannya* (jika false jadi true, jika true jadi false). Ketika state berubah, React langsung **menggambar ulang layar seketika (Re-render)** tanpa me-refresh browser!

Pola Pengiriman Form: try dan catch

```
const handleLogin = async (e) => {
  e.preventDefault();
  try {
    // Coba minta izin masuk ke Backend Laravel
    await authApi.login(emailInput, passwordInput);
    toast.success("Login Berhasil!");
    navigate("/dashboard"); // Buka pintu dashboard
  } catch (err) {
    // Tangkap penolakan dari Backend jika password salah
    toast.error("Login Gagal! Username atau Password salah.");
  }
};
```

BAB 7: Glosarium Istilah Penting Fullstack Developer

Berikut kamus istilah populer dunia kerja yang kini sudah Anda kuasai maknanya:

Istilah	Artinya dalam Bahasa Manusia Awam
Frontend	Semua yang terlihat di layar monitor browser pengguna (dibangun dengan React & Tailwind).
Backend	Dapur logika dan keamanan server di balik layar yang tidak terlihat pengguna (dibangun dengan Laravel).
API (Application Programming Interface)	Pelayan kurir perantara yang mengantar paket data bolak-balik antara Frontend dan Backend via internet.
JSON	Format nota standar bertukar data berupa teks terstruktur rapi, contoh: <code>{"key": "nama", "status": "aktif"}</code> .
Database (RDBMS)	Gudang penyimpanan data permanen di harddisk yang menghubungkan tabel-tabel bersaudara.
Migration	Kertas cetak biru kode untuk merancang tabel dan kolom database secara otomatis.
Model (ORM)	Juru bicara perwakilan tabel di database dalam bahasa pemrograman PHP (Eloquent).
Controller	Koki pengatur alur kerja: menerima data dari API, memvalidasi aturan, dan menyimpannya ke database.
Validation	Pemeriksaan kelayakan data di pintu gerbang (contoh: nama wajib diisi, maksimal 255 karakter).
Authorization (Otorisasi)	Pemeriksaan hak akses pengguna (siapa yang boleh membuka atau mengubah data tertentu).
State	Ingatan jangka pendek komponen di browser yang membuat layar reaktif dan interaktif seketika.
Multi-Tenancy	Arsitektur satu aplikasi yang dipakai bersama banyak sekolah tanpa data mereka saling bocor.

Istilah	Artinya dalam Bahasa Manusia Awam
Soft Deletes / nullOnDelete	Teknik mengamankan rekam jejak data agar tidak lenyap saat data induknya dihapus demi keperluan audit.

Selamat! Anda kini telah menguasai fondasi arsitektur Fullstack Developer profesional.

Dokumen ini dicetak otomatis oleh sistem SIMMACI • Lembaga Pendidikan Ma'arif NU Cilacap